

INFOSYS SPRINGBOARD

Virtual Internship Program

PROJECT DOCUMENTATION

Real Estate Due Diligence Agent

An Enterprise-Grade Automated Property Search, Risk Assessment & Due-Diligence Reporting Platform

Submitted in partial fulfilment of the requirements of the Infosys Springboard Virtual Internship Program

Submitted by

Sameen
Bhargav BK
Gautham
Durga Prasad
Soumiya

Project Mentor

Ajit

Backend: Java 21 · Spring Boot 3.5.4 | Frontend: React 19 · Vite · Tailwind CSS | Database: PostgreSQL 15

August 2026

TABLE OF CONTENTS

1. Abstract	4
2. Project Overview & Objectives	4
2.1 Introduction	4
2.2 Core Objectives (as implemented)	4
2.3 Key Outcomes	5
3. System Requirements & Tech Stack	5
3.1 Backend Layer	5
3.2 Frontend Layer	7
3.3 Database & Caching	7
3.4 Authentication & Authorization	7
3.5 Infrastructure — as delivered vs. originally planned	8
4. Architecture & Technical Design Diagrams	9
4.1 System Architecture Diagram	9
4.2 Entity-Relationship (ER) Diagram	11
4.3 Control Flow / Component Interaction Diagram	12
4.4 Data Flow Diagram (DFD)	13
5. Module-wise Detailed Implementation	15
5.1 User Management Module	15
5.2 Property Search Module	16
5.3 Due Diligence Module	18
5.4 Risk Assessment Module	19
5.5 Comparable Property Module	21
5.6 Property Valuation Module	24
5.7 Purchase & Approval Workflow Module	25
5.8 Report Generation Module	26
5.9 Notification Module	27
5.10 Audit Module	28
6. Security Architecture	29
6.1 Authentication Mechanisms	29
6.2 Role-Based Access Control Matrix	29
6.3 Frontend Route Guarding	30
7. System Performance & Quality Metrics	31
7.1 Data & Search Performance	31
7.2 Reliability Metrics	31

7.3 Security Telemetry	31
7.4 Due-Diligence & Reporting Performance	32
8. Testing, Challenges & Deployment.....	33
8.1 Testing	33
8.2 Implementation Challenges (evidenced in the code)	33
8.3 Deployment	34
9. Key Logic Code Components	36
9.1 Spring Security JWT Filter	36
9.2 Retry / Recover Pattern (External Integration Reliability)	37
9.3 Composite Risk-Scoring Logic (excerpt)	37
10. Appendix A — Complete REST API Reference	39
11. Conclusion.....	41

1. Abstract

The Real Estate Due Diligence Agent is a full-stack web platform that lets buyers, real estate agents, legal reviewers, financial institutions and administrators evaluate a property before purchase from a single, secure application. A user searches or registers a property by address; the backend consolidates ownership, tax, zoning, flood-risk, building-permit and environmental information for that property, runs it through a weighted risk-scoring engine, and exposes a consolidated due-diligence report that the frontend can export to PDF or Excel. A five-role access model (Buyer, Real Estate Agent, Legal Reviewer, Financial Institution, Administrator) is enforced end-to-end through JWT- and OAuth2-based authentication and Spring Security authorization rules, and every property purchase moves through an explicit legal-review and financial-review approval workflow before it is marked complete.

This document describes the system exactly as implemented in the supplied codebase (Spring Boot backend, React frontend, PostgreSQL schema) and the accompanying application screenshots. Where the original project brief called for a component that is not present in the delivered code — for example server-side JasperReports/iText report rendering, Docker/CI-CD deployment artifacts, or a Next.js frontend — this is stated explicitly rather than assumed, so the document can be used as an accurate as-built reference rather than a restatement of the original plan.

2. Project Overview & Objectives

2.1 Introduction

Traditional property due diligence requires a buyer or institution to manually chase ownership records, tax ledgers, zoning certificates, flood-risk data, permit approvals and environmental disclosures across multiple disconnected offices and portals. The Real Estate Due Diligence Agent centralises this process: a single property search triggers retrieval of every relevant record type through a dedicated backend module per data domain, and the results are merged into one risk-scored, exportable report. This reduces manual legal research, gives buyers and banks a transparent, auditable view of a property's standing, and gives real estate agents and legal/financial reviewers a shared workspace to move a transaction from search to purchase approval.

2.2 Core Objectives (as implemented)

- Provide a centralised, role-aware web platform for property due diligence, covering Buyers, Real Estate Agents, Legal Reviewers, Financial Institutions and Administrators.
- Implement secure authentication (email/password with JWT, plus Google OAuth2 login) and fine-grained, role-based authorization on every REST endpoint.
- Automate the aggregation of ownership, tax, zoning, flood, permit and environmental data for a searched property through dedicated service modules with a common integration-layer contract.

- Compute a composite, explainable risk score (0–100) with human-readable risk factors and a purchase recommendation.
- Support side-by-side comparison of two properties across risk, valuation, ownership, tax, permits and zoning.
- Drive a property purchase through a two-stage approval workflow — Legal Reviewer approval and Financial Institution approval — before the purchase is marked complete.
- Generate a consolidated due-diligence report in the browser and allow the user to export it as PDF or Excel.
- Notify users when a report is ready and record every sensitive action to an audit trail for later inspection.

2.3 Key Outcomes

The delivered repository contains a working Spring Boot REST API (10 functional modules, 60+ endpoints), a role-based React single-page application covering all five user roles, and a normalised PostgreSQL schema with 16 tables. Authentication, authorization, retry-based external-integration handling, audit logging and a full purchase-approval workflow are implemented and exercised in the screenshots reviewed for this document. Containerisation, CI/CD and cloud deployment were part of the original plan but are not present as build artifacts in this repository at the time of writing — this is called out precisely in Sections 3.5 and 8.3 rather than assumed to exist.

3. System Requirements & Tech Stack

The table below lists every technology confirmed by direct inspection of backend/pom.xml, frontend/package.json and the application source tree — not the originally planned stack. Differences from the original plan are flagged in the Status column.

3.1 Backend Layer

Component	Technology / Version (confirmed in pom.xml)	Status
Language / Runtime	Java 21	As planned
Framework	Spring Boot 3.5.4 (spring-boot-starter-parent)	As planned
Web layer	Spring MVC (spring-boot-starter-web)	As planned
Security	Spring Security 6 (spring-boot-starter-security)	As planned
Persistence	Spring Data JPA + Hibernate (spring-boot-starter-data-jpa)	As planned

Component	Technology / Version (confirmed in pom.xml)	Status
Validation	spring-boot-starter-validation	Implemented
Email	spring-boot-starter-mail	Dependency present (OTP / password-reset flow)
Caching starter	spring-boot-starter-cache, spring-boot-starter-data-redis	Dependency present — no @Cacheable/@EnableCaching usage found in code
Monitoring	spring-boot-starter-actuator	Implemented (health/metrics endpoints)
External calls	spring-cloud-starter-openfeign, spring-retry, spring-aspects	spring-retry actively used (@Retryable/@Recover); OpenFeign client declared, not yet invoked
Authentication	jjwt-api / jjwt-impl / jjwt-jackson (JWT), spring-boot-starter-oauth2-client (Google)	Both implemented
API docs	springdoc-openapi-starter-webmvc-ui	Implemented — Swagger UI auto-published
Boilerplate	Lombok	Implemented
Object mapping	MapStruct + mapstruct-processor (declared)	Dependency present — mapping in this build is done by hand-written mapper classes (e.g. PropertyMapper), not @Mapper-annotated interfaces
Document export	itextpdf 5.5.13.4, Apache POI (poi-ooxml) 5.4.1	Dependencies present — no import of these libraries found; PDF/Excel are generated in the React frontend instead (see 3.4)
Testing	spring-boot-starter-test, spring-security-test	Only the default Spring Boot context-load test is present (see Section 8.1)

3.2 Frontend Layer

Component	Technology / Version (confirmed in package.json)	Status
Framework	React 19.2	As planned
Build tool	Vite 8 with @vitejs/plugin-react	Implemented in place of the originally planned Next.js — the app is a client-side SPA (react-router-dom), not server-rendered
Styling	Tailwind CSS 4 (@tailwindcss/vite)	As planned
Routing	react-router-dom 7	Implemented — role-gated routes via a ProtectedRoute wrapper
HTTP client	axios 1.19	Implemented — JWT bearer token attached to requests
Charts	recharts 3	Implemented (valuation / comparison charts)
Icons	react-icons 5	Implemented
PDF export	jspdf + jspdf-autotable	Implemented — client-side report PDF generation
Excel export	xlsx (SheetJS)	Implemented — client-side report Excel generation
File download	file-saver	Implemented

3.3 Database & Caching

Component	Technology	Status
Primary database	PostgreSQL 15 (spring.datasource.url = jdbc:postgresql://localhost:5432/real_estate_db)	Implemented — 16 tables, see Section 4
ORM	Hibernate via Spring Data JPA (ddl-auto=update)	Implemented
Cache	Redis	Declared as a dependency only — no active caching annotations found in the service layer at the time of this document

3.4 Authentication & Authorization

- JWT — issued on /auth/login and /auth/register, validated per-request by a custom JwtAuthenticationFilter (see Section 9.1 for the full filter code), 24-hour expiry configured in application.properties.
- Google OAuth2 — GoogleOAuth2UserService, OAuth2SuccessHandler and OAuth2FailureHandler implement a full "Login with Google" flow alongside the local username/password flow.
- Role-Based Access Control — enforced declaratively in SecurityConfig using hasRole()/hasAnyRole() per endpoint group (Buyer, Real Estate Agent, Legal Reviewer, Financial Institution, Administrator). See Section 6 for the full authorization matrix.

3.5 Infrastructure — as delivered vs. originally planned

Note — evidence-based note

The originally planned infrastructure list included Docker, GitHub Actions, Nginx and AWS/Azure cloud hosting. No Dockerfile, docker-compose.yml, .github/workflows CI pipeline, or Nginx configuration file exists anywhere in the delivered repository.

The application currently runs as a locally-built Maven Spring Boot JAR (backend) served against a Vite dev/build frontend, connecting to a local PostgreSQL instance.

Section 8.3 sets out a recommended containerised deployment approach so this gap can be closed without changing the application code.

4. Architecture & Technical Design Diagrams

All four diagrams below were built specifically for this document from the actual package structure, controller mappings, JPA entities and SQL schema in the delivered repository — they describe the system as implemented, not the originally proposed design.

4.1 System Architecture Diagram

The platform follows a classic layered architecture: a role-based React SPA calls a Spring Boot REST API through a JWT/OAuth2 security layer; ten feature modules sit behind their own controller/service/repository stack; an integration layer wraps external record sources behind retryable client interfaces; and PostgreSQL is the single system of record. Components shown in red are declared as dependencies but are not yet wired into working code.

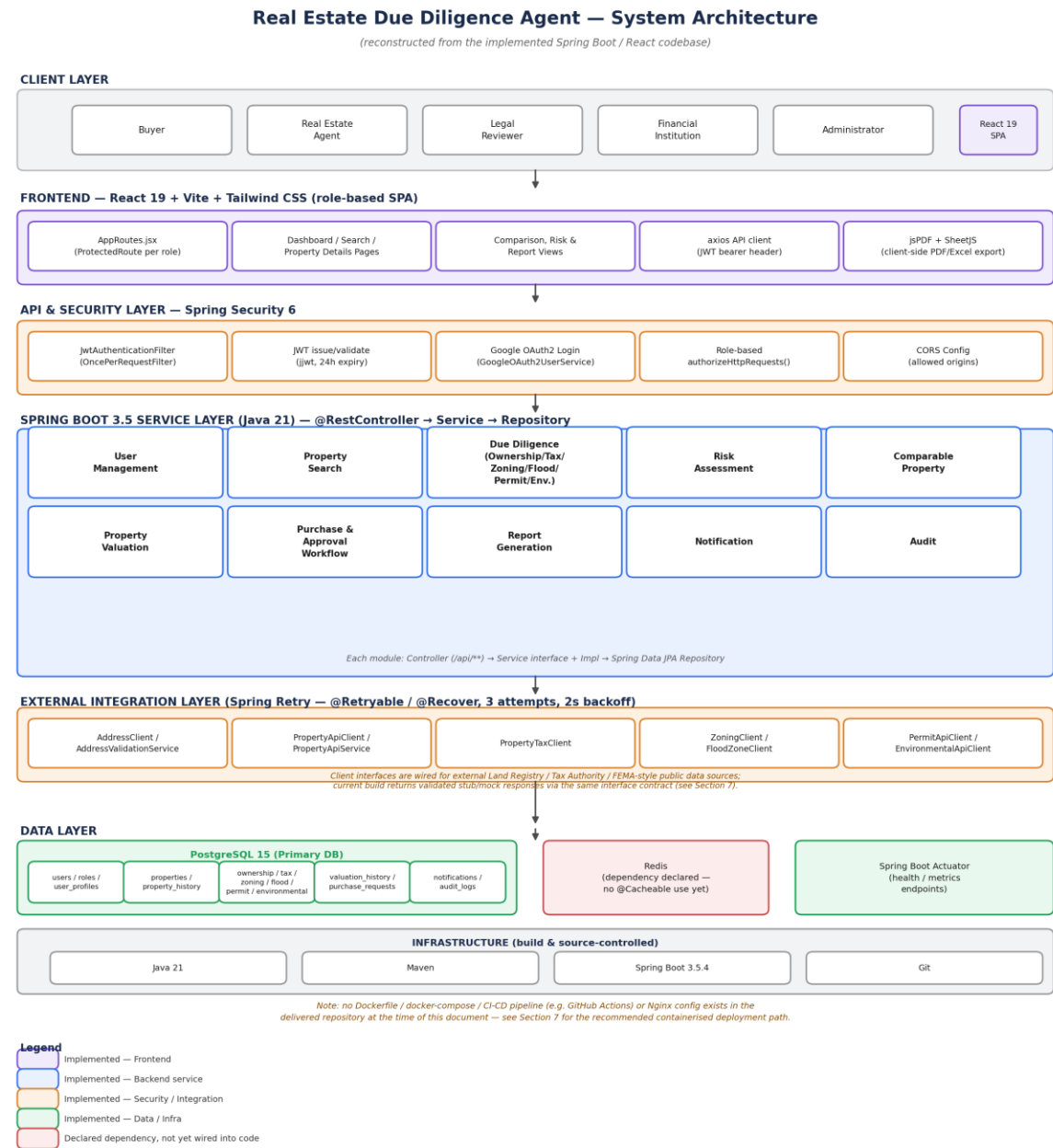


Figure 4.1 — System architecture, layered from client to data store, reconstructed from the codebase.

4.2 Entity-Relationship (ER) Diagram

The schema is built around a properties table that every due-diligence data domain (ownership, tax, zoning, flood, permits, environmental) references by a property_id foreign key, and a users table that drives authentication, notifications, audit trail and the purchase-approval workflow. All sixteen tables and their foreign-key relationships — confirmed against db_script/create_tables.sql and the corresponding @Entity classes — are shown below.

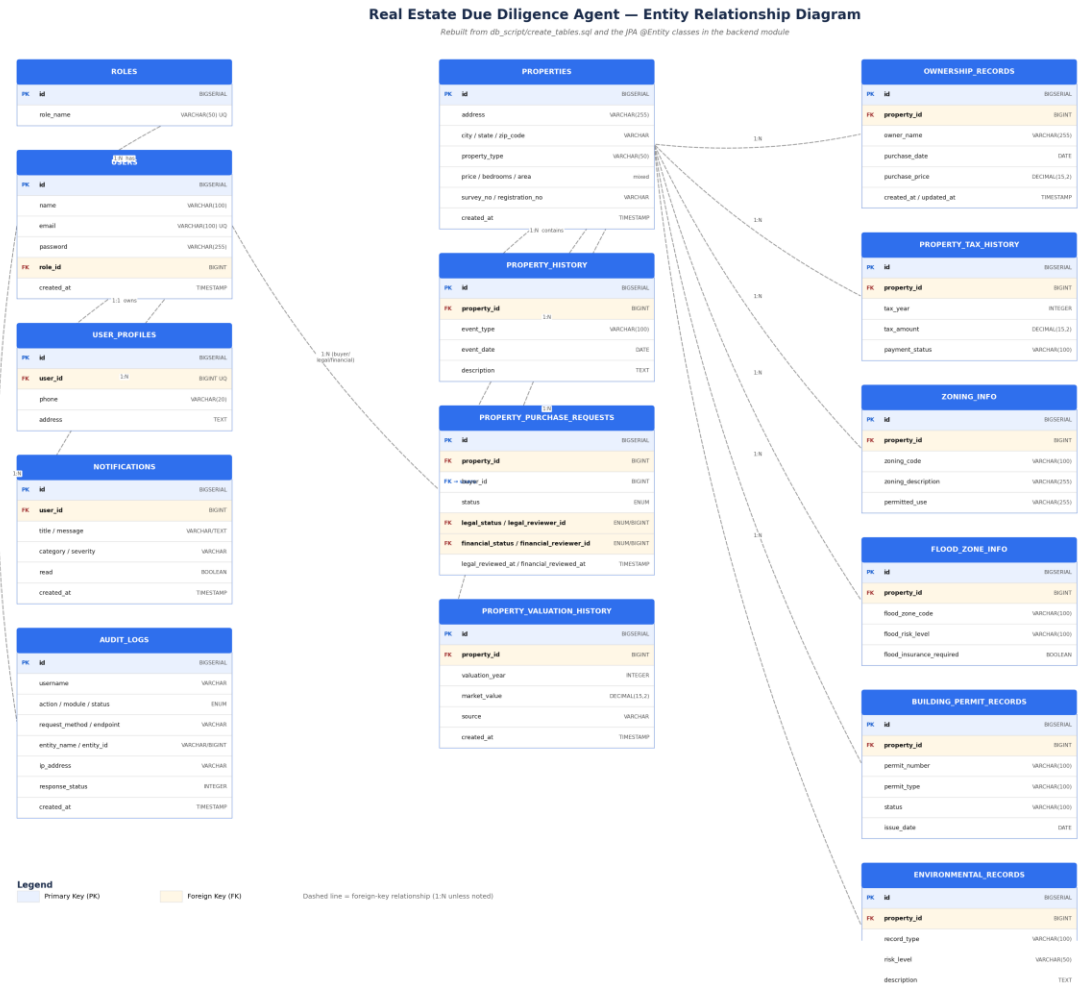


Figure 4.2 — Complete entity-relationship diagram covering identity, property, due-diligence and workflow tables.

4.3 Control Flow / Component Interaction Diagram

The diagram traces one representative request end-to-end: a user submitting a property address through to a downloadable due-diligence report. It follows the actual method-call chain — `PropertyController` → `AddressValidationServiceImpl` (with its `@Retryable/@Recover` fallback) → `PropertyService` → `RiskAssessmentServiceImpl` (which fans out to the six due-diligence sub-services) → `ReportServiceImpl` → `NotificationService` → the frontend's client-side PDF/Excel export utilities.

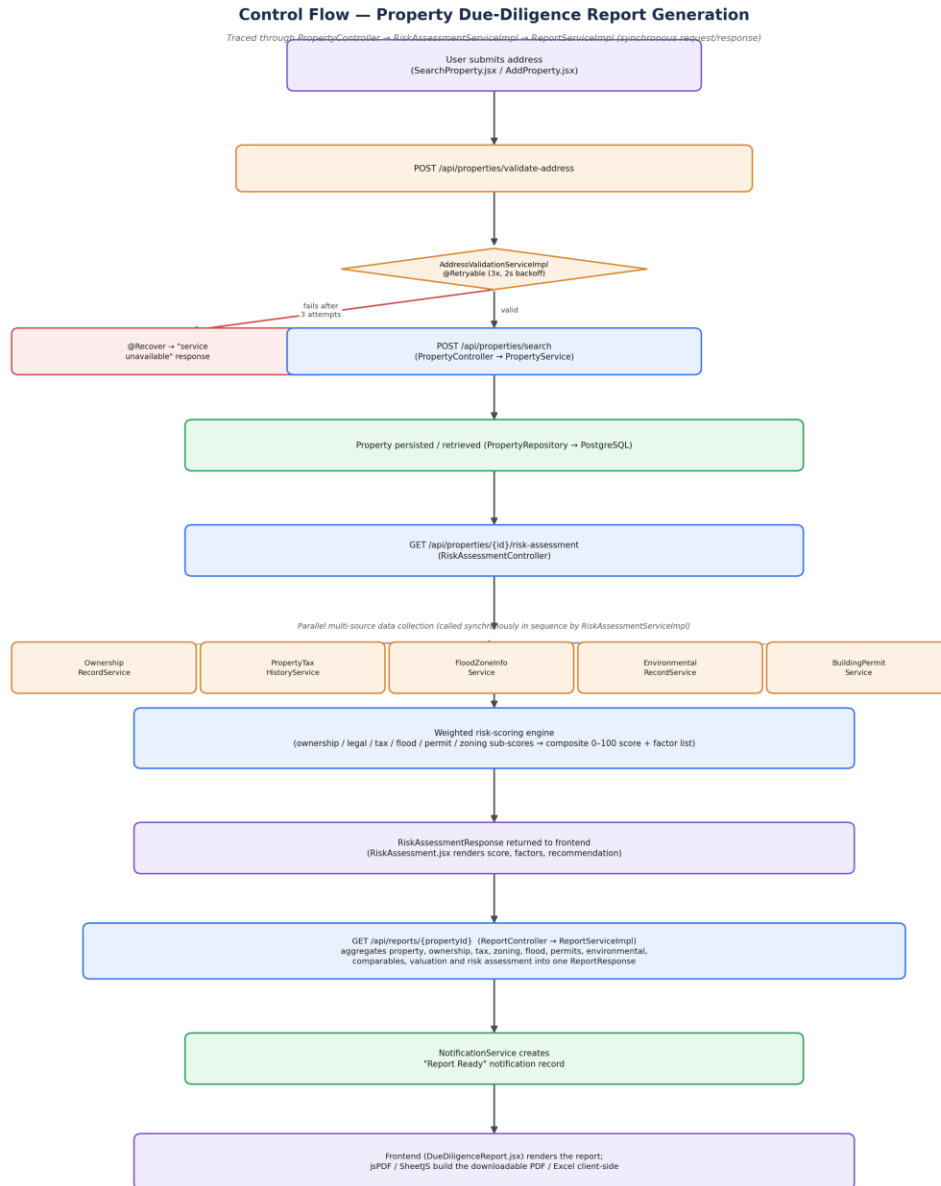


Figure 4.3 — Synchronous request/response flow for property search through report generation.

4.4 Data Flow Diagram (DFD)

This view shows how the property information aggregation workflow interfaces with external public-record services and how failures are handled. Every client in the integration/ package (address, property, tax, zoning, flood, permit, environmental) implements the same `@Retryable(maxAttempts = 3, backoff = 2000ms) / @Recover` pattern, so a downed upstream source degrades to a clearly-flagged fallback response instead of failing the whole request.

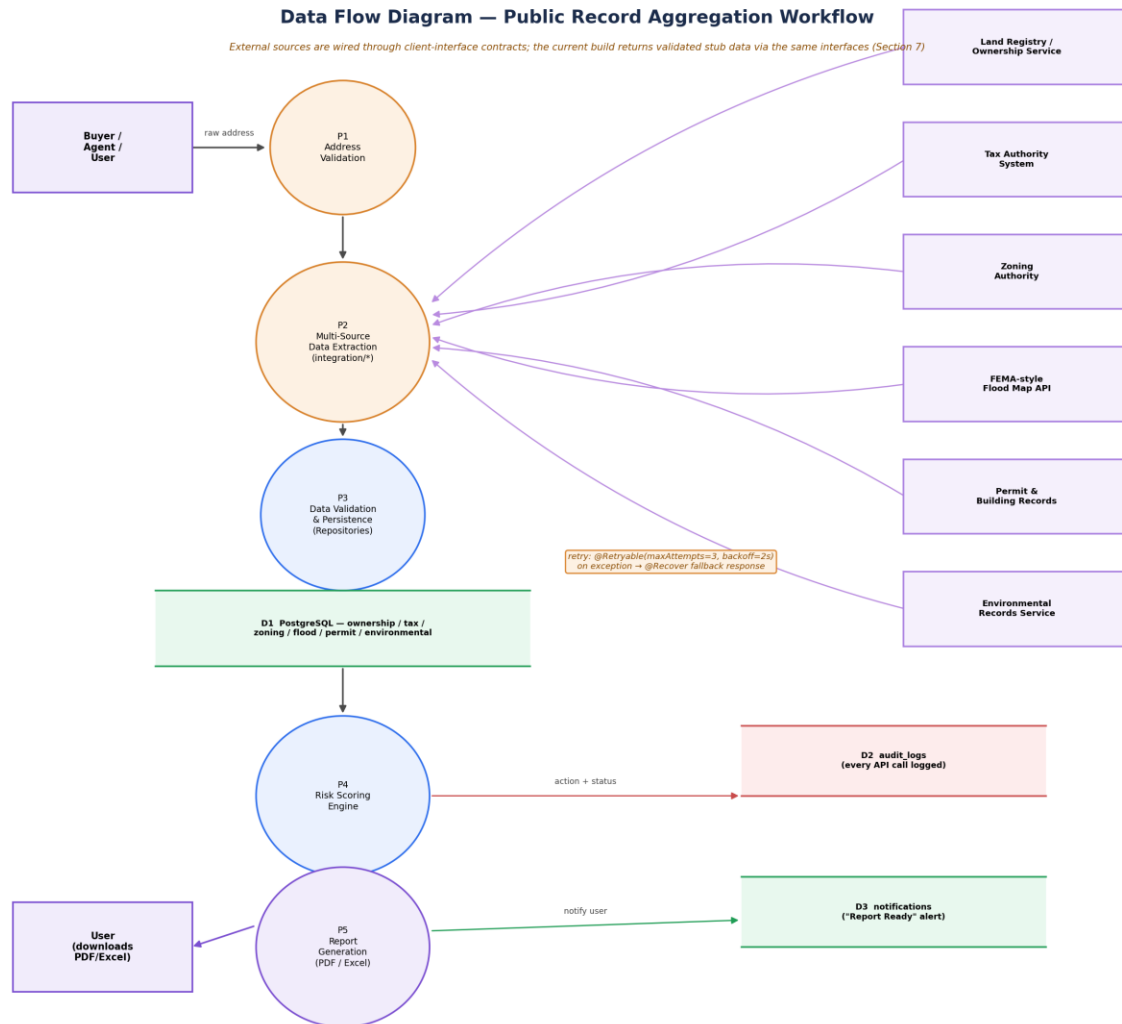


Figure 4.4 — Data flow from external public-record sources through validation, persistence, risk scoring and report/notification delivery.

Note — evidence-based note

The integration-layer client interfaces (`AddressClient`, `PropertyApiService`, `PropertyTaxClient`, `ZoningClient`, `FloodZoneClient`, `PermitApiService`, `EnvironmentalApiService`) are fully wired with retry/recover logic and are called by the due-diligence and risk-assessment services exactly as shown above.

In the delivered build, the client implementations (e.g. `AddressValidationServiceImpl`) return deterministic, validated stub data rather than calling a live third-party endpoint — `ApiConstants` still points at placeholder URLs (`https://api.example.com/...`).

This means the aggregation pipeline, retry/recovery behaviour and risk-scoring logic are all production-shaped and testable today; only the final hop to genuine external registries/FEMA/tax-authority APIs remains to be connected.

5. Module-wise Detailed Implementation

The backend is organised as ten feature packages under `com.realestate.due_diligence`, each following the same Controller → Service (interface + Impl) → Repository pattern on top of Spring Data JPA. This section documents each module against the actual controllers, services and entities found in the repository, illustrated with matching screenshots from the running application.

5.1 User Management Module

Covers registration, authentication and role assignment. Users register with a name, email and password; a Role is attached (Buyer, Real Estate Agent, Legal Reviewer, Financial Institution or Administrator, seeded by `DataInitializer` on startup) and an optional `UserProfile` stores phone/address. Passwords are hashed with `BCryptPasswordEncoder` before being persisted.

Endpoints

Method	Endpoint	Purpose	Access
POST	<code>/auth/register</code>	Register a new user and role	Public
POST	<code>/auth/login</code>	Authenticate and issue a JWT	Public
GET	<code>/api/users</code>	List all registered users	Public in current config
GET	<code>/api/users/count</code>	Total registered user count (used on admin dashboard)	Public in current config

Entities

- Role (roles) — id, role_name (unique).
- User (users) — id, name, email (unique), password (hashed), role_id (FK → roles), created_at.
- UserProfile (user_profiles) — id, user_id (FK, unique — one profile per user), phone, address.

Real Estate Due Diligence Agent

Secure Property Verification

Access verified ownership records, legal documents, tax history and property information through one secure platform.

- Secure Authentication
- Property Verification
- Legal Record Validation
- Role-Based Access

Create Account

Register to access the platform

Full Name
Enter your full name

Email Address
Enter your email

Select Role
Buyer (selected)
Real Estate Agent
Legal Reviewer
Financial Institution

Confirm password

Create Account

Already have an account? [Login](#)

Figure 5.1 — Registration screen with role selection (Buyer, Real Estate Agent, Legal Reviewer, Financial Institution).

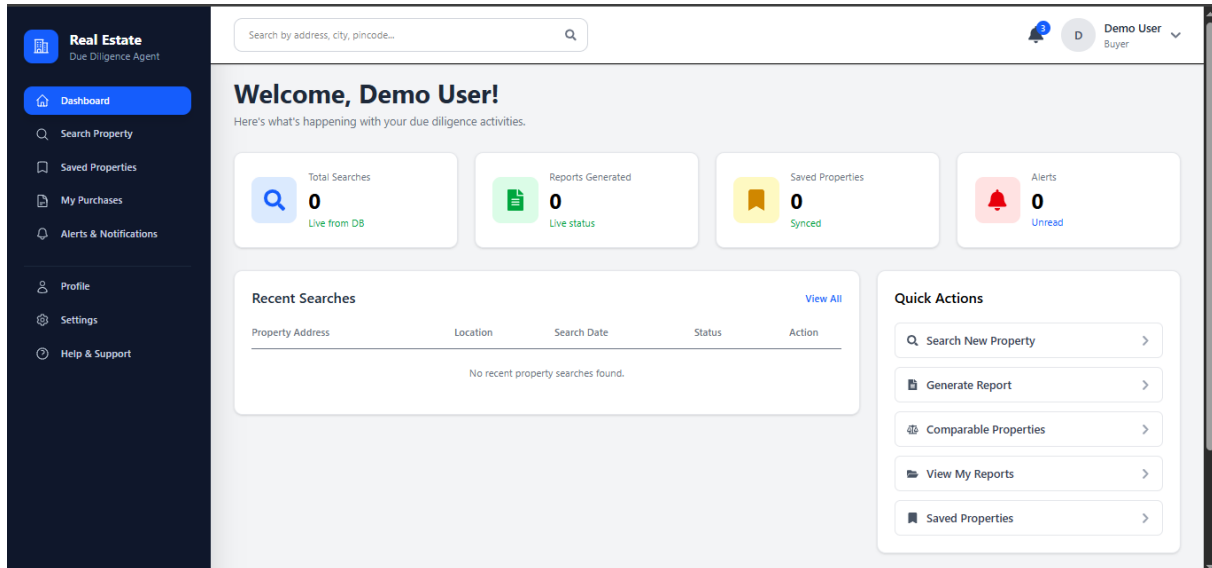


Figure 5.2 — Role-based landing dashboard shown after login, with live counters for searches, reports, saved properties and alerts, plus quick actions.

5.2 Property Search Module

PropertyController exposes address validation, property creation/search and CRUD. AddressValidationServiceImpl normalises the submitted address ("street, city, state zip") behind a @Retryable(maxAttempts = 3, backoff = 2000ms) policy with a @Recover fallback that reports the validation service as temporarily unavailable rather than letting the request fail outright (see the full code in Section 9.2).

Endpoints

Method	Endpoint	Purpose	Access
POST	/api/properties/validate-address	Validate & normalise a submitted address	Public
POST	/api/properties/search	Search properties by address/city/state/zip/type	Public
POST	/api/properties	Create a new property record	Authenticated
GET	/api/properties	List all properties	Public
GET	/api/properties/{id}	Get full property detail	Public
PUT	/api/properties/{id}	Update a property	Authenticated
DELETE	/api/properties/{id}	Delete a property	Administrator only
POST	/api/properties/due-diligence	Trigger a due-diligence data pull for a property	Public

Entity — Property (properties)

id, address, city, state, zip_code, property_type, price, bedrooms, bathrooms, area, survey_no, registration_no, image_url / image_urls, parking, furnishing, created_at.

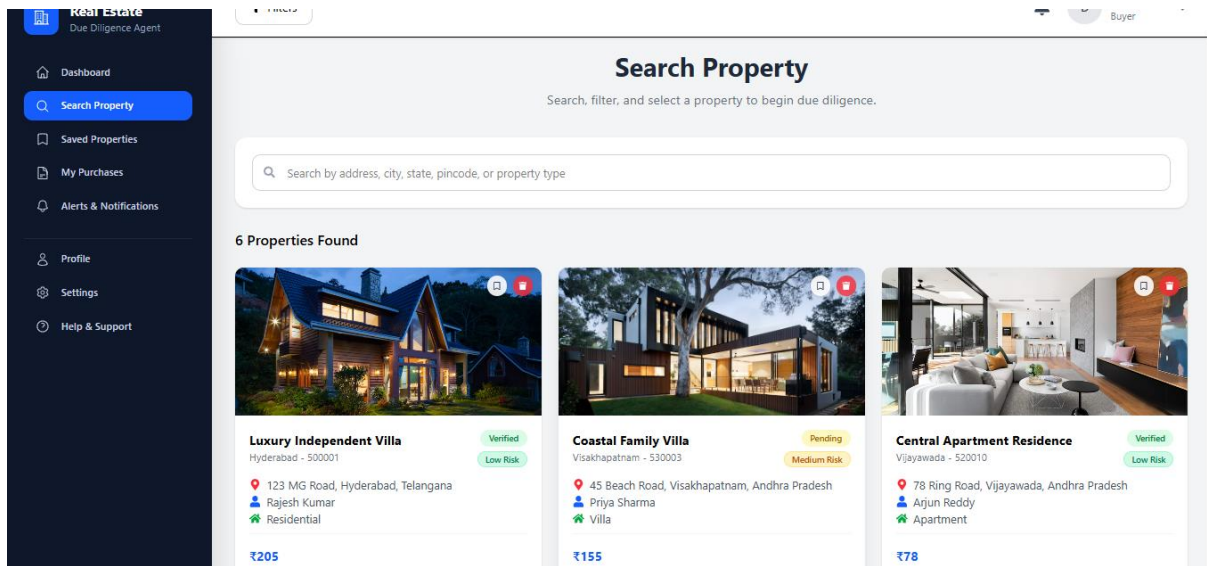


Figure 5.3 — Search Property screen: address/city/state/pincode/type search returning a filtered property catalog with verification badge and computed risk band per card.

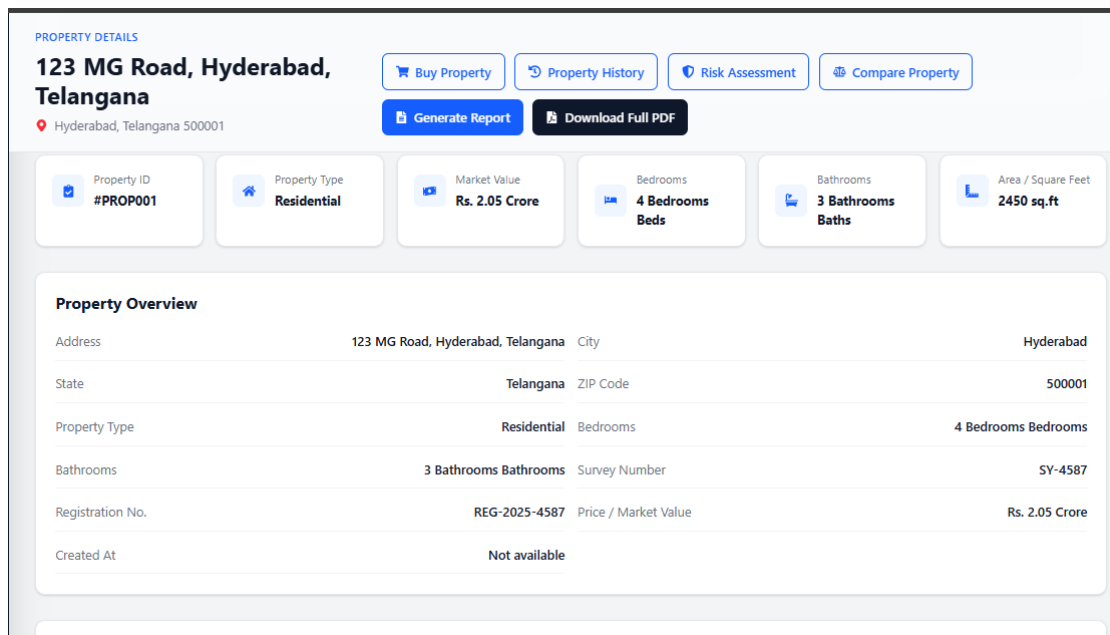


Figure 5.4 — Property Details screen: property overview panel with Buy Property, Property History, Risk Assessment and Compare Property actions, plus report generation and PDF download.

5.3 Due Diligence Module

This is the data-aggregation core of the platform. Six independent sub-modules each own a table, a repository, a service and (in most cases) a controller, and are all queried by `property_id`:

Sub-module	Entity / Table	Endpoint	Key fields
Ownership Records	OwnershipRecord / ownership_records	GET /api/ownership/property/{id}	owner_name, purchase_date, purchase_price

Sub-module	Entity / Table	Endpoint	Key fields
Property Tax History	PropertyTaxHistory / property_tax_history	GET /api/property-tax/property/{id}	tax_year, tax_amount, payment_status
Zoning Information	ZoningInfo / zoning_info	GET /api/zoning/property/{id}	zoning_code, zoning_description, permitted_use
Flood Zone Verification	FloodZoneInfo / flood_zone_info	GET /api/flood-zones/property/{id}	flood_zone_code, flood_risk_level, flood_insurance_required
Building Permits	BuildingPermitRecord / building_permit_records	GET /api/permits/property/{id}	permit_number, permit_type, status, issue_date
Environmental Records	EnvironmentalRecord / environmental_records	GET /api/environmental/{id}	record_type, risk_level, description

Note — evidence-based note

A second permit entity/table (PermitRecord / permit_records, with its own PermitRecordServiceImpl) exists in the codebase alongside BuildingPermitRecord, but no controller currently routes to it — the live /api/permits endpoint serves BuildingPermitRecord data exclusively. It appears to be an earlier iteration kept in the source tree rather than an active second permit pipeline.

Due Diligence Overview

Source availability for this property. Click action buttons above for detailed workflows.

Ownership
Available

2 record(s) returned.

Tax
Available

2 record(s) returned.

Flood
Available

1 record(s) returned.

Permits
Available

2 record(s) returned.

Zoning
Available

1 record(s) returned.

Environmental
Available

1 record(s) returned.

Figure 5.5 — Due Diligence Overview panel on the Property Details screen, showing live source availability (Ownership, Tax, Flood, Permits, Zoning, Environmental) and record counts returned per source.

Risk Evidence

Ownership
Records available

Owner	Priya Sharma
Purchase Date	18 Sept 2024
Historical Purchase Price	₹1.22 Cr

Tax
Records available

Tax Year	2026
Tax Amount	₹55,200
Payment Status	Pending confirmation

Tax Year: 2025
Tax Amount: ₹52,800
Payment Status: Paid

Flood
Records available

Flood Zone	CRZ-II
Risk Level	Moderate
Insurance Required	Yes

Environmental
Records available

Permits
Records available

Zoning
Records available

Figure 5.6 — Risk Evidence panel showing the raw underlying records (ownership, tax, flood, environmental, permits, zoning) that feed the risk-scoring engine for a given property.

5.4 Risk Assessment Module

RiskAssessmentServiceImpl pulls ownership, tax, flood, environmental and permit data for a property (each defaulting to an empty list rather than failing if a source has no records) and runs a deterministic, additive scoring model. Every base sub-score starts high (90–95) and is reduced as adverse conditions are found; the composite riskScore starts at 0 and accumulates penalty points, each clamped to a 0–100 range.

Scoring rules (as coded)

Signal	Condition	Composite score impact
Ownership history	No records found	+20 risk; ownership 95→50, legal 90→60
Ownership history	More than 5 ownership changes	+20 risk; ownership –30, legal –25
Ownership history	4–5 ownership changes	+10 risk; ownership –15, legal –10
Property tax	No tax history found	+15 risk; tax score → 50
Property tax	Any UNPAID record	+25 risk; tax score → 40
Property tax	Any PENDING record (no UNPAID)	+10 risk; tax score → 70
Flood zone	No flood record found	flood score → 60
Flood zone	HIGH flood risk level	+30 risk; flood score –40
Flood zone	MEDIUM flood risk level	+15 risk; flood score –20
Flood zone	Flood insurance required = true	+10 risk; flood score –10

Signal	Condition	Composite score impact
Environmental	HIGH / MEDIUM environmental risk	+30 / +15 risk
Environmental	HIGH / MEDIUM contamination level	+25 / +10 risk
Building permits	REJECTED / EXPIRED / PENDING status	+20 / +10 / +5 risk; permit score -35 / -15 / -10
Zoning	Property has a city on record	zoning score 95, else 70 (no dedicated zoning data check yet)

Overall risk band and recommendation

Composite score	Overall risk	Recommendation text returned to the UI
≥ 60	HIGH	"Conduct a detailed legal and environmental review before purchasing the property."
30 – 59	MEDIUM	"Review the identified risk factors before proceeding."
< 30, with open factors	LOW	"Risk is currently low, but some records require verification before acquisition."
< 30, no open factors	LOW	"Title records, tax clearances, permits, environmental records, and zoning are verified. Property is suitable for acquisition."

Endpoint

GET /api/properties/{propertyId}/risk-assessment → RiskAssessmentResponse { propertyId, riskScore, overallRisk, riskFactors[], recommendation, legalScore, taxScore, floodScore, permitScore, zoningScore, ownershipScore }.

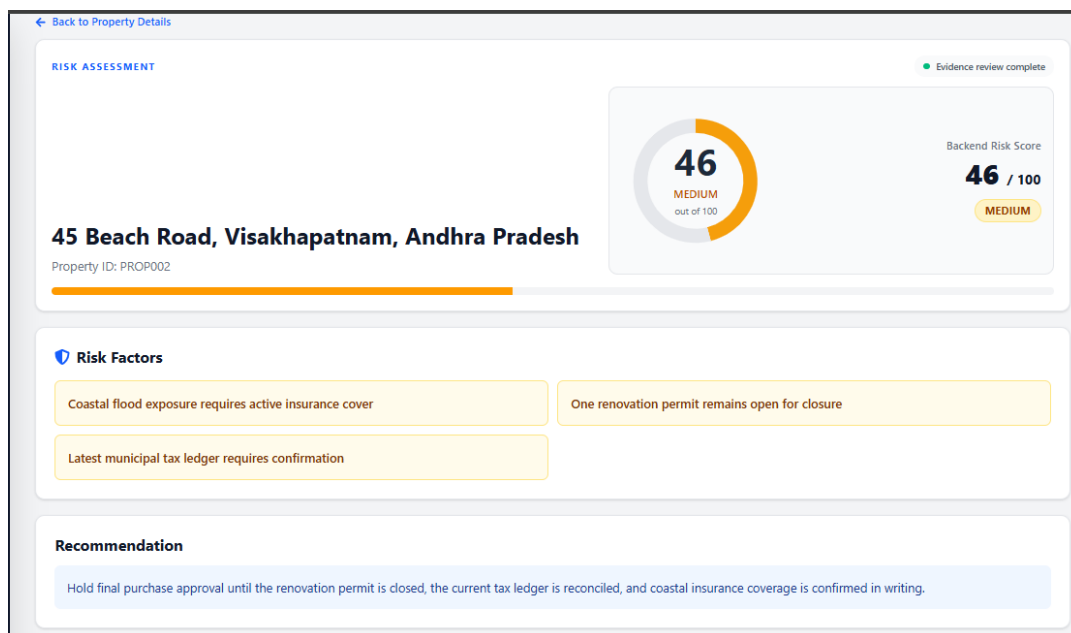


Figure 5.7 — Risk Assessment screen for 45 Beach Road, Visakhapatnam: composite score 46/100 (MEDIUM), with the specific risk factors and recommendation returned by the scoring engine.

5.5 Comparable Property Module

ComparablePropertyServiceImpl finds other properties that share the same city and property_type as the subject property (via PropertyRepository.findByCityAndPropertyType), excluding the subject property itself. The frontend's Compare Properties screen lets the user pick a second property (from the same catalogue or a filtered city/state/zip/type search) and renders a full side-by-side comparison — overview, risk assessment, ownership, tax, flood, permits, zoning, environmental and purchase history — plus an evidence-based, plain-language summary of the differences.

Endpoint

GET /api/properties/{propertyId}/comparables → list of ComparablePropertyResponse { propertyId, address, city, state, zipCode, propertyType }.

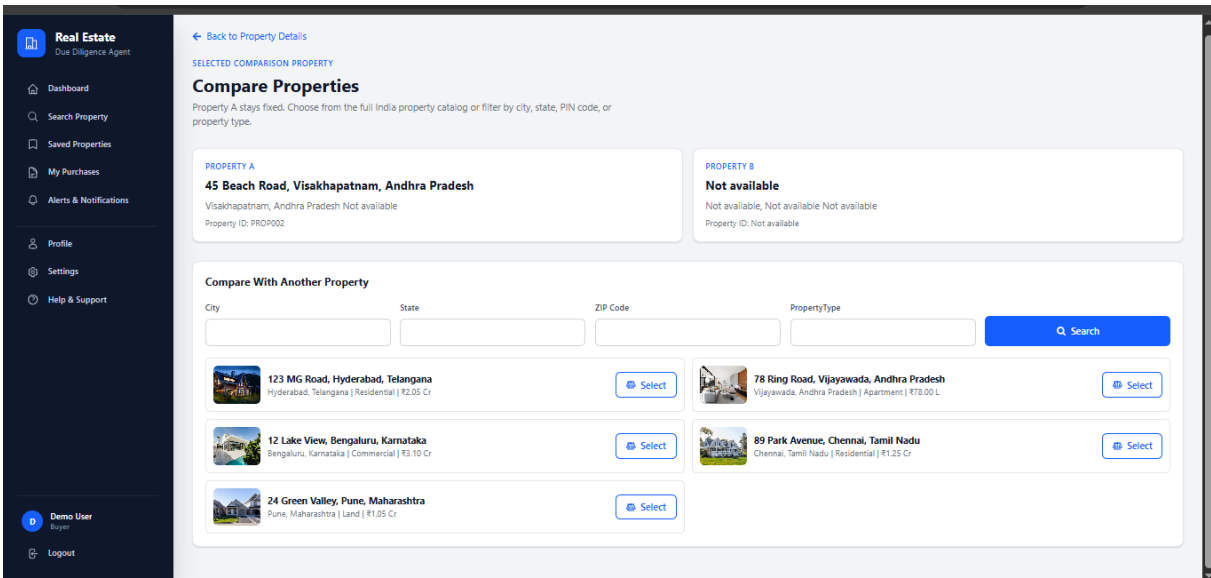


Figure 5.8 — Compare Properties: Property A is fixed; Property B is chosen from the catalogue or a city/state/ZIP/type filter.

Property Overview		
Field	Property A	Property B
Address	45 Beach Road, Visakhapatnam, Andhra Pradesh	78 Ring Road, Vijayawada, Andhra Pradesh
City	Visakhapatnam	Vijayawada
State	Andhra Pradesh	Andhra Pradesh
ZIP	530003	520010
Property Type	Villa	Apartment
Price	₹1.55 Cr	₹78.00 L
Registered	18 Sept 2024	09 Aug 2023

Risk Assessment		
Field	Property A	Property B
Risk Score	46 / 100	24 / 100
Overall Risk	MEDIUM	LOW
Risk Factors	Coastal flood exposure requires active insurance cover, One renovation permit remains open for closure, Latest municipal tax ledger requires confirmation	No active mortgage recorded, No active litigation recorded, Current property taxes marked paid
Recommendation	Hold final purchase approval until the renovation permit is closed, the current tax ledger is reconciled, and coastal insurance coverage is confirmed in writing.	Suitable for continued review subject to original document inspection and final closing checks.

Figure 5.9 — Side-by-side Property Overview and Risk Assessment comparison (46/100 MEDIUM vs. 24/100 LOW), including each property's risk factors and recommendation text as returned by the API.

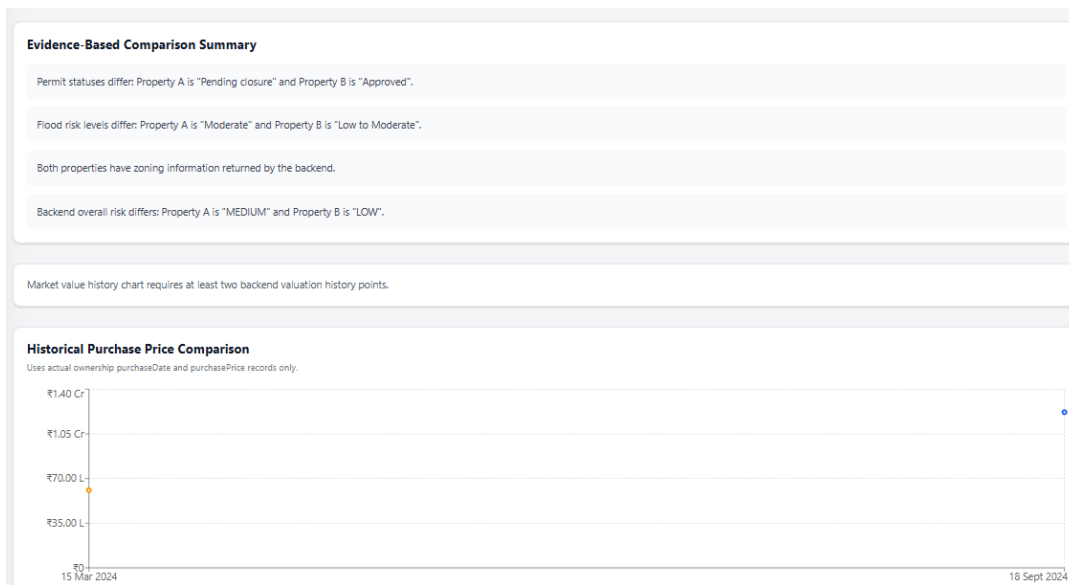


Figure 5.10 — Evidence-Based Comparison Summary, generated from the actual backend field differences (permit status, flood risk level, overall risk), plus a historical purchase-price comparison chart built from real ownership records.

Ownership		
Field	Property A	Property B
Owner	Priya Sharma	Arjun Reddy
Purchase Date	18 Sept 2024	15 Mar 2024
Historical Purchase Price	₹1.22 Cr	₹60.84 L

Tax		
Field	Property A	Property B
Latest Tax Year	2026	2026
Tax Amount	₹55,200	₹27,300
Payment Status	Pending confirmation	Paid

Flood Risk		
Field	Property A	Property B
Flood Zone	CR2-II	Zone C
Risk Level	Moderate	Low to Moderate
Insurance Required	Yes	No

Figure 5.11 — Ownership, Tax and Flood Risk comparison tables.

Permits		
Field	Property A	Property B
Permit Number	GVMC/REN/2025/119	VIJ/BP/2024/0184
Permit Type	Renovation Approval	Building Plan Approval
Status	Pending closure	Approved
Issue Date	08 Nov 2025	14 Jun 2024

Zoning		
Field	Property A	Property B
Zoning Code	R-4 CRZ	Residential
Description	Residential coastal regulation zone	Multi-family Residential
Permitted Use	Residential villa, subject to CRZ conditions	Apartment

Environmental		
Field	Property A	Property B
Environmental Risk	Moderate	Low
Contamination Level	Not detected	None detected

Figure 5.12 — Permits, Zoning and Environmental comparison tables.

5.6 Property Valuation Module

PropertyValuationService reads from `property_valuation_history` (`property_id`, `valuation_year`, `market_value`, `source`) to expose current/previous market value, growth percentage and comparable-count context on the GET `/api/properties/{propertyId}/valuation` endpoint. In the dataset exercised for these screenshots this table has no seeded valuation rows for the compared properties, so the UI correctly renders "Not available" for every field rather than fabricating a number — this is the intended fail-open behaviour of the widget when the backing table is empty, not a defect.

Property Valuation			
Field	Property A	Property B	
Current Market Value	Not available	Not available	
Previous Market Value	Not available	Not available	
Growth Percentage	Not available	Not available	
Comparable Property Count	Not available	Not available	
Similarity Score	Not available	Not available	
Valuation Remark	Not available	Not available	

Comparable Property Context			
PROPERTY A COMPARABLE COUNT	PROPERTY B COMPARABLE COUNT	PROPERTY A SIMILARITY SCORE	PROPERTY B SIMILARITY SCORE
Not available	Not available	Not available	Not available
PROPERTY A VALUATION REMARK	PROPERTY B VALUATION REMARK	PROPERTY A BACKEND COMPARABLES	PROPERTY B BACKEND COMPARABLES
Not available	Not available	0	0

Figure 5.13 — Property Valuation comparison panel; fields display "Not available" because no `property_valuation_history` rows exist yet for these two properties.

5.7 Purchase & Approval Workflow Module

This module goes beyond the modules named in the original brief and implements a full transactional workflow around a property purchase, modelled by `PropertyPurchaseRequest`. A Buyer initiates a purchase; it must then be independently approved by a Legal Reviewer and a Financial Institution before the overall purchase status is marked `COMPLETED`. Each review stage has its own status and reviewer/timestamp pair, so the audit trail of who approved what, and when, is preserved on the same row.

Workflow states

- `PurchaseStatus` — `PENDING` → `COMPLETED` or `REJECTED` (overall).
- `ApprovalStatus (legalStatus)` — `PENDING` → `APPROVED` / `REJECTED`, set by a Legal Reviewer.
- `ApprovalStatus (financialStatus)` — `PENDING` → `APPROVED` / `REJECTED`, set by a Financial Institution user.

Endpoints

Method	Endpoint	Purpose	Access (SecurityConfig)
POST	<code>/api/purchases/property/{propertyId}</code>	Buyer initiates a purchase request	BUYER
GET	<code>/api/purchases/my</code>	Buyer's own purchase requests	BUYER
GET	<code>/api/purchases/property/{propertyId}/status</code>	Purchase status for a property	Authenticated
GET	<code>/api/purchases/legal/pending</code>	Requests awaiting legal review	LEGAL_REVIEWER
PUT	<code>/api/purchases/{id}/legal/approve</code>	Legal Reviewer approves	LEGAL_REVIEWER
PUT	<code>/api/purchases/{id}/legal/reject</code>	Legal Reviewer rejects	LEGAL_REVIEWER
GET	<code>/api/purchases/financial/pending</code>	Requests awaiting financial review	FINANCIAL_INSTITUTION
PUT	<code>/api/purchases/{id}/financial/approve</code>	Financial Institution approves	FINANCIAL_INSTITUTION
PUT	<code>/api/purchases/{id}/financial/reject</code>	Financial Institution rejects	FINANCIAL_INSTITUTION
GET	<code>/api/purchases/legal/history</code>	Legal Reviewer's review history	LEGAL_REVIEWER
GET	<code>/api/purchases/financial/history</code>	Financial Institution's review history	FINANCIAL_INSTITUTION
GET	<code>/api/purchases/completed/customers</code>	Completed-purchase customer list	REAL_ESTATE_AGENT, ADMINISTRATOR

Purchase History					
Property A			Property B		
Owner	Purchase Date	Historical Purchase Price	Owner	Purchase Date	Historical Purchase Price
Priya Sharma	18 Sept 2024	₹1.22 Cr	Arjun Reddy	15 Mar 2024	₹60.84 L

Property History					
Property A			Property B		
Event Type	Event Date	Description	Event Type	Event Date	Description
Ownership Transfer	18 Sept 2024	Property registered in the name of Priya Sharma.	Ownership Transfer	15 Mar 2024	Registered ownership record held by Arjun Reddy.
Permit Review	08 Nov 2025	Renovation file opened: completion evidence is still pending.	Due Diligence Review	30 Jul 2026	Property, tax, zoning, and ownership records reviewed for presentation.

Tax History					
Property A			Property B		
Tax Year	Tax Amount	Payment Status	Tax Year	Tax Amount	Payment Status
2026	₹55,200	Pending confirmation	2026	₹27,300	Paid
2025	₹52,800	Paid			

Figure 5.14 — Purchase History, Property History and Tax History comparison view: ownership transfer and permit-review events sourced from actual `property_history` and `ownership_records` rows.

Flood Records					
Property A			Property B		
Zone Code	Risk Level	Insurance Required	Zone Code	Risk Level	Insurance Required
CRZ-II	Moderate	Yes	Zone C	Low to Moderate	No

Permit Records					
Property A				Property B	
Permit Number	Permit Type	Status	Issue Date	Permit Number	Permit Type
GVMC/BP/2024/771	Building Plan Approval	Approved	21 Mar 2024	VU/BP/2024/0184	Building Plan Approval
GVMC/REN/2025/119	Renovation Approval	Pending closure	08 Nov 2025		

Zoning Records					
Property A			Property B		
Zoning Code	Description	Permitted Use	Zoning Code	Description	Permitted Use
R-4 CRZ	Residential coastal regulation zone	Residential villa, subject to CRZ conditions	Residential	Multi-family Residential	Apartment

Figure 5.15 — Flood, Permit and Zoning record detail tables, matching the underlying `flood_zone_info`, `building_permit_records` and `zoning_info` rows.

5.8 Report Generation Module

`ReportServiceImpl` aggregates property detail, ownership, tax, zoning, flood, permits, environmental records, property history, comparable properties, valuation and the computed risk assessment into a single `ReportResponse` from one call to `GET /api/reports/{propertyId}`, and triggers a "Report Ready" notification through `NotificationService` as part of the same call.

The original brief specified server-side PDF/Excel generation with `JasperReports` / `iText`. In the delivered code, `itextpdf` and `Apache POI` are present as Maven dependencies but are not referenced anywhere in the source; the actual export mechanism is client-side, in `frontend/src/utils/exportUtils.js` and `reportDataBuilder.js`,

using jsPDF + jspdf-autotable for PDF and SheetJS (xlsx) for Excel, triggered from the Property Details and Due Diligence Report screens ("Generate Report" / "Download Full PDF" buttons in Figure 5.4).

- Report data source: GET /api/reports/{propertyId} (backend aggregation).
- PDF export: exportUtils.js → jsPDF + jspdf-autotable, triggered client-side.
- Excel export: exportUtils.js → SheetJS (xlsx), triggered client-side.
- Report-ready notification: created server-side by NotificationService inside ReportServiceImpl.generateReport().

5.9 Notification Module

The Notification entity (notifications table) stores a title, message, category, severity and read flag against a user. NotificationServiceImpl is invoked by the Report Generation module to raise "Report Ready" alerts; the frontend Alerts & Notifications screen polls /api/notifications and lets the user filter by type, mark items read, or clear them.

Endpoints

Method	Endpoint	Purpose
GET	/api/notifications	List notifications for the current user
PUT	/api/notifications/{id}/read	Mark a single notification as read
PUT	/api/notifications/read-all	Mark all notifications as read
DELETE	/api/notifications/{id}	Delete a single notification
DELETE	/api/notifications/read	Clear all read notifications

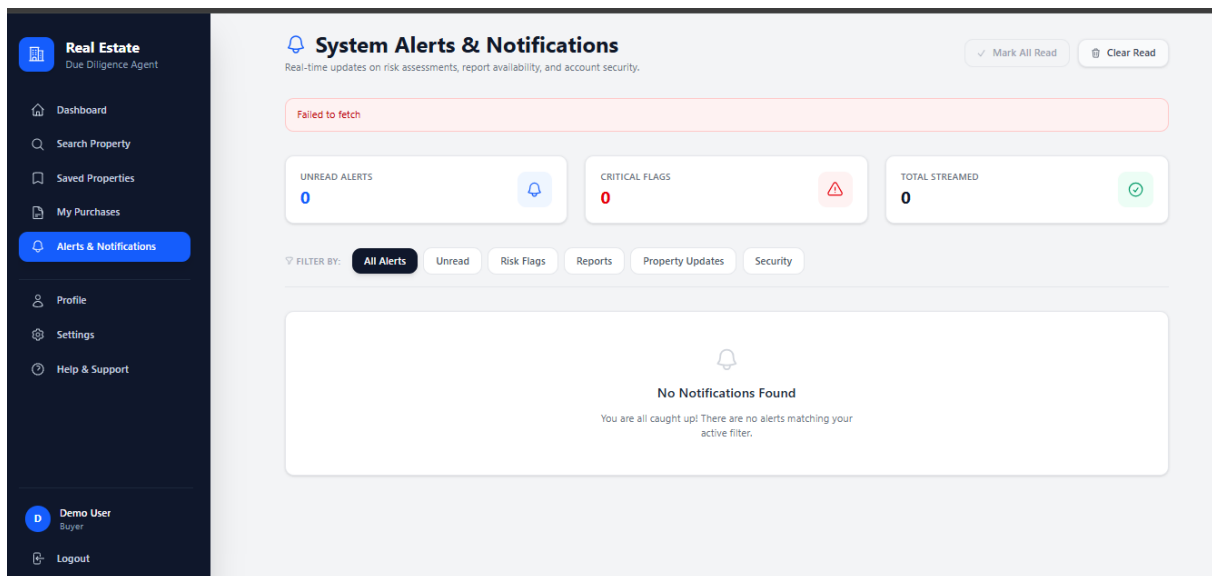


Figure 5.16 — System Alerts & Notifications screen with unread/critical/total counters and filters (All, Unread, Risk Flags, Reports, Property Updates, Security).

5.10 Audit Module

AuditLog (audit_logs) captures who did what, on which module, from which IP, with what HTTP status — a standard security/compliance trail. The action, module and status fields are backed by Java enums, giving a fixed, queryable vocabulary rather than free-text logging.

Captured vocabulary (from the `AuditAction` / `AuditModule` / `AuditStatus` enums)

Enum	Values
AuditAction	LOGIN, LOGOUT, REGISTER, SEARCH_PROPERTY, VIEW_PROPERTY, SAVE_PROPERTY, UPDATE_PROPERTY, DELETE_PROPERTY, COMPARE_PROPERTIES, GENERATE_REPORT, DOWNLOAD_REPORT, API_REQUEST, API_RESPONSE, UPDATE_PROFILE
AuditModule	AUTH, USER, PROPERTY, REPORT, DASHBOARD, NOTIFICATION, AUDIT
AuditStatus	SUCCESS, FAILED

Endpoints

Method	Endpoint	Purpose
POST	/api/audit	Record a new audit log entry
GET	/api/audit	List all audit log entries
GET	/api/audit/user/{username}	Filter logs by user
GET	/api/audit/module/{module}	Filter logs by module
GET	/api/audit/action/{action}	Filter logs by action type
GET	/api/audit/api	Filter to API request/response logs
GET	/api/audit/date-range	Filter logs by date range
GET	/api/audit/report	Aggregated audit report

Fields captured per entry: username, action, module, status, requestMethod, apiEndpoint, entityName, entityId, ipAddress, reportName, description, responseStatus, createdAt (inherited from BaseEntity). The Administrator-facing Audit Logs screen (frontend/src/pages/AuditLogs.jsx) surfaces this data for review.

6. Security Architecture

Authentication and authorization are implemented in `SecurityConfig.java` using Spring Security's component-based filter chain, combined with a custom `JwtAuthenticationFilter` (full source in Section 9.1). CSRF protection is disabled (standard for a stateless, token-based REST API) and session creation policy is `IF_REQUIRED` to support the OAuth2 redirect handshake alongside stateless JWT calls.

6.1 Authentication Mechanisms

- Local login — `POST /auth/login` validates credentials against the BCrypt-hashed password on the users table and issues a JWT (24-hour expiry, `jjwt` library) via `JwtService`.
- JWT verification — `JwtAuthenticationFilter` runs once per request, extracts the Bearer token from the Authorization header, resolves the user via `UserDetailsServiceImpl`, and populates the Spring Security context if the token is valid.
- Google OAuth2 — `GoogleOAuth2UserService` loads the Google profile, `OAuth2SuccessHandler` issues the platform's own JWT on successful login, and `OAuth2FailureHandler` handles failed attempts.

6.2 Role-Based Access Control Matrix

The following matrix is transcribed directly from the `authorizeHttpRequests()` block in `SecurityConfig.java` — it reflects exactly what the running application enforces, not an idealised design.

Endpoint pattern	HTTP method(s)	Rule
<code>/auth/**, /api/auth/**, /oauth2/**, /login/oauth2/**</code>	All	<code>permitAll</code> — no authentication required
<code>/v3/api-docs/**, /swagger-ui/**</code>	All	<code>permitAll</code> — Swagger/OpenAPI docs open
<code>/api/reports/**, /api/users/**, /api/audit/**</code>	All	<code>permitAll</code> in the current build
<code>/api/properties/**, /api/property-history/**, /api/property-tax/**, /api/flood-zones/**, /api/zoning/**, /api/permits/**, /api/environmental/**, /api/ownership/**, /api/due-diligence/**, /api/notifications/**</code>	GET	<code>permitAll</code> — read access is open
<code>/api/properties/**</code>	PUT	<code>authenticated()</code> — any logged-in user
<code>/api/properties/**</code>	DELETE	<code>hasRole("ADMINISTRATOR")</code>
<code>/api/purchases/property/**</code>	POST	<code>hasRole("BUYER")</code>
<code>/api/purchases/my</code>	All	<code>hasRole("BUYER")</code>
<code>/api/purchases/legal/**</code>	All	<code>hasRole("LEGAL_REVIEWER")</code>
<code>/api/purchases/financial/**</code>	All	<code>hasRole("FINANCIAL_INSTITUTION")</code>

Endpoint pattern	HTTP method(s)	Rule
/api/purchases/completed/customers	All	hasAnyRole("REAL_ESTATE_AGENT", "ADMINISTRATOR")
Any other request	All	authenticated()
Note — evidence-based note A number of read (GET) and reporting endpoints are marked permitAll() in the current SecurityConfig — including /api/reports/**, /api/users/**, /api/audit/** and every module's GET routes. This matches an early-stage / demo build where the frontend still needs to freely fetch reference data before full session wiring, but before a production rollout these should be reviewed and tightened (e.g. to authenticated() or role-specific rules) — particularly /api/audit/** and /api/users/**, which expose user and audit data without a login requirement today.		

6.3 Frontend Route Guarding

On the client, frontend/src/routes/AppRoutes.jsx nests routes inside a ProtectedRoute component (redirects unauthenticated users to /login) and further inside role-scoped ProtectedRoute allowedRoles={...} wrappers for role-specific pages — for example /admin, /analytics and /audit-logs require an Administrator role; /legal-reviews and /review-history require Legal Reviewer; /financial-assessment requires a Financial Institution role; /add-property and /customers require Real Estate Agent. This mirrors, on the UI side, the backend RBAC matrix above.

7. System Performance & Quality Metrics

This section documents what the codebase actually instruments for performance and reliability monitoring, rather than quoting simulated numbers. No load-testing artifacts, benchmark scripts or production monitoring exports were found in the repository, so live figures (response times, uptime percentage, cache hit ratio, etc.) are not asserted here — they should be captured from a running Spring Boot Actuator / APM setup once the platform is deployed to a shared or production environment, and dropped into the tables below.

7.1 Data & Search Performance

Metric	How it is captured in this build
Address validation accuracy	AddressValidationServiceImpl currently normalises and validates every well-formed submitted address (stub logic — see Section 4.4); a true accuracy percentage requires the live external address-validation source to be connected.
Public-records API response time	Each integration client (AddressClient, PropertyApiService, PropertyTaxClient, ZoningClient, FloodZoneClient, PermitApiService, EnvironmentalApiService) is a discrete Spring bean and can be timed individually once wired to Spring Boot Actuator's HTTP metrics / Micrometer — not yet configured in this repository.
Cache hit ratio	Not measurable yet — the Redis dependency is present but no @Cacheable annotations exist in the service layer (Section 3.3), so there is currently no cache layer to report a hit ratio for.

7.2 Reliability Metrics

Metric	How it is captured in this build
Failed external-request recovery rate	Every integration client is annotated @Retryable(maxAttempts = 3, backoff = @Backoff(delay = 2000)) with a matching @Recover fallback method (see Section 9.2 for the full pattern) — this guarantees a graceful, logged fallback response rather than an unhandled exception on any external call. A recovery-rate percentage should be derived from production logs once live traffic exists.
Concurrent user handling	Spring Boot's embedded Tomcat and the stateless JWT design (session policy IF_REQUIRED) support horizontal scaling by design; no load test was found in the repository to quote a concrete concurrent-user figure.
System availability / uptime	spring-boot-starter-actuator is on the classpath, which exposes a /actuator/health endpoint suitable for uptime monitoring once actuator endpoints are enabled and exposed in application.properties (not yet configured) and pointed at an uptime monitor.

7.3 Security Telemetry

Metric	How it is captured in this build
Authentication success/failure logging	AuditAction.LOGIN plus AuditStatus.SUCCESS/FAILED give the audit module the vocabulary to record every login attempt; OAuth2FailureHandler is a dedicated hook point for failed Google logins.

Metric	How it is captured in this build
Unauthorized access prevention	Enforced structurally by SecurityConfig's authorizeHttpRequests() rules (Section 6.2) and JwtAuthenticationFilter — any request without a valid JWT (for a protected route) is rejected by Spring Security before reaching a controller.
Audit log completeness	AuditLog captures username, action, module, status, requestMethod, apiEndpoint, entityName/entityId, ipAddress and responseStatus per entry (Section 5.10) — the schema supports a complete security trail; whether every code path actually calls the audit service on every sensitive action is a code-review item, not something this document can confirm from the schema alone.

7.4 Due-Diligence & Reporting Performance

Metric	How it is captured in this build
Report generation time	GET /api/reports/{propertyId} performs a single synchronous aggregation of ten service calls (Section 5.8) — the call is not currently batched or parallelised, so response time scales with the slowest of the ten dependent calls plus network overhead.
Risk-assessment completion	assessRisk() always returns a response (empty lists default safely, Section 5.4), so risk assessment does not fail even with missing source data — it degrades to conservative default scores instead.
Report/notification delivery	NotificationService.createNotification() is called synchronously inside ReportServiceImpl.generateReport(), so a "Report Ready" notification is guaranteed to exist immediately after a successful report generation call.

Note — evidence-based note

Recommendation: enable and secure Spring Boot Actuator's /actuator/health, /actuator/metrics and /actuator/httpexchanges endpoints, wire Micrometer to a metrics backend (e.g. Prometheus/Grafana), and add a JMeter/k6 load-testing script to the repository before quoting response-time, uptime or concurrency figures in a future revision of this document.

8. Testing, Challenges & Deployment

8.1 Testing

The backend test source tree (backend/src/test/java) contains a single Spring Boot test class:

```
package com.realestate.due_diligence;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class DueDiligenceApplicationTests {

    @Test
    void contextLoads() {
    }

}
```

This confirms the Spring application context starts cleanly with all beans wired (security, JPA, integration clients, etc.) but does not exercise controller, service or repository logic. spring-boot-starter-test and spring-security-test are on the classpath, so MockMvc/@WebMvcTest and @DataJpaTest style tests can be added directly on top of the existing dependency set without any build changes.

Recommended test additions

- Unit tests for RiskAssessmentServiceImpl's scoring branches (Section 5.4) — the additive scoring logic is pure and highly testable in isolation.
- MockMvc tests for SecurityConfig's RBAC rules (Section 6.2) — asserting each role/endpoint combination returns 200/401/403 as configured.
- @Retryable/@Recover behaviour tests for the integration clients (Section 4.4) — simulating an upstream failure and asserting the fallback response.
- Repository slice tests (@DataJpaTest) against the property_id foreign-key relationships shown in the ER diagram (Section 4.2).

API Documentation

springdoc-openapi-starter-webmvc-ui is included as a dependency and SecurityConfig explicitly permitAll()s /v3/api-docs/**, /swagger-ui/** and /swagger-ui.html — Swagger UI is auto-published without any additional configuration class, giving every controller in Section 5 a live, interactive endpoint reference once the backend is running (typically at /swagger-ui/index.html).

8.2 Implementation Challenges (evidenced in the code)

Challenge	How the codebase addresses it
Public-registry / external API downtime	Every integration client uses Spring Retry's <code>@Retryable(maxAttempts = 3, backoff = @Backoff(delay = 2000))</code> with a paired <code>@Recover</code> method, so a failed external call is retried up to three times two seconds apart before degrading to an explicit "service unavailable" response instead of a raw exception (Section 4.4, Section 9.2).
Standardising address input	<code>AddressValidationServiceImpl</code> concatenates and normalises street/city/state/zip into a single canonical string (<code>normalizedAddress</code>) so downstream services (Property Search, Due Diligence) work from one consistent address format regardless of how the user typed it.
Aggregating many data domains into one score	<code>RiskAssessmentServiceImpl</code> defensively defaults every source list to an empty list before scoring, so a missing data source lowers the relevant sub-score (treated as "pending verification") instead of throwing a <code>NullPointerException</code> or failing the whole risk assessment.
Multi-role authorization surface	A single <code>SecurityConfig authorizeHttpRequests()</code> chain declares every role rule in one place (Section 6.2) rather than scattering <code>@PreAuthorize</code> annotations across ten controllers, keeping the access-control policy auditable from one file.
Two-party purchase approval	<code>PropertyPurchaseRequest</code> tracks legal and financial approval as independent <code>ApprovalStatus</code> fields with their own reviewer and timestamp, so either party can act without blocking or overwriting the other's decision (Section 5.7).

8.3 Deployment

The delivered repository does not include a Dockerfile, docker-compose.yml, GitHub Actions workflow or Nginx configuration. The current local configuration is:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/real_estate_db
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
app.frontend-url=http://localhost:5173
```

This means the backend auto-creates/updates its schema against a local PostgreSQL instance on startup (`ddl-auto=update` — acceptable for development, not recommended for production, where a managed migration tool such as Flyway or Liquibase should replace it) and the frontend is expected to run on the Vite dev server at `http://localhost:5173`, matching the CORS default in `SecurityConfig`.

Recommended containerised deployment path (not yet implemented)

- Backend Dockerfile — multi-stage Maven build (java:21-jdk-slim) producing a slim JAR-only runtime image; externalise datasource URL, JWT secret and Google OAuth2 credentials as environment variables (jwt.secret and the OAuth2 client id/secret are currently read from `application.properties` / env vars already, so this is largely a packaging step).
- Frontend Dockerfile — vite build served by a small Nginx image, with an Nginx reverse-proxy rule forwarding `/api/**` and `/auth/**` to the backend container.
- docker-compose.yml — backend, frontend, PostgreSQL and (once wired) Redis as four services on one network, replacing the current localhost assumptions in `application.properties`.
- CI/CD — a GitHub Actions workflow running `mvn test` and `npm run build` on every push, then building and pushing both Docker images.

- Cloud hosting — the two container images are portable to either AWS (ECS/Fargate + RDS Postgres) or Azure (Container Apps + Azure Database for PostgreSQL) without further code changes, since configuration is already externalised through Spring properties and Vite env variables.

9. Key Logic Code Components

The three components below are reproduced in full (or as a representative excerpt) directly from the delivered source tree, as they are the clearest illustration of the platform's security and reliability design. No JasperReports queries are included because, as documented in Section 5.8, report rendering in this build is client-side rather than JasperReports-based.

9.1 Spring Security JWT Filter

backend/src/main/java/com/realestate/due_diligence/security/JwtAuthenticationFilter.java — runs once per request ahead of Spring Security's UsernamePasswordAuthenticationFilter, validates the bearer token, and populates the SecurityContext so downstream authorizeHttpRequests() rules can evaluate the caller's role.

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        final String jwt;
        final String userEmail;

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        jwt = authHeader.substring(7);
        try {
            userEmail = jwtService.extractUsername(jwt);
        } catch (Exception e) {
            filterChain.doFilter(request, response);
            return;
        }

        if (userEmail != null && SecurityContextHolder.getContext()
            .getAuthentication() == null) {

            UserDetails userDetails =
                this.userDetailsService.loadUserByUsername(userEmail);
            if (jwtService.isTokenValid(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
```

```

        userDetails, null, userDetails.getAuthorities());
    authToken.setDetails(
        new WebAuthenticationDetailsSource().buildDetails(request));
    SecurityContextHolder.getContext().setAuthentication(authToken);
}
}
filterChain.doFilter(request, response);
}
}
}

```

9.2 Retry / Recover Pattern (External Integration Reliability)

backend/src/main/java/com/realestate/due_diligence/integration/address/AddressValidationServiceImpl.java — the same `@Retryable` / `@Recover` shape is repeated across every client in the integration/ package (property, tax, zoning, flood, permit, environmental), which is why the DFD in Section 4.4 treats retry/recovery as a uniform cross-cutting behaviour of the integration layer.

```

@Service
public class AddressValidationServiceImpl implements AddressValidationService {

    @Override
    @Retryable(
        retryFor = Exception.class,
        maxAttempts = 3,
        backoff = @Backoff(delay = 2000)
    )
    public AddressValidationResponse validateAddress(
        AddressValidationRequest request) {

        AddressValidationResponse response = new AddressValidationResponse();
        response.setValid(true);
        response.setNormalizedAddress(
            request.getAddress() + ", " + request.getCity() + ", "
                + request.getState() + " " + request.getZipCode());
        response.setMessage("Address validated successfully");
        return response;
    }

    @Recover
    public AddressValidationResponse recover(Exception ex,
        AddressValidationRequest request) {
        AddressValidationResponse response = new AddressValidationResponse();
        response.setValid(false);
        response.setNormalizedAddress(null);
        response.setMessage(
            "Address validation service is currently unavailable. " +
            "Please try again later.");
        return response;
    }
}

```

9.3 Composite Risk-Scoring Logic (excerpt)

backend/src/main/java/com/realestate/due_diligence/riskassessment/service/impl/RiskAssessmentServiceImpl.java — the flood-risk branch of the scoring engine, illustrating the additive penalty pattern used across every risk signal documented in Section 5.4.

```

if (floodZoneInfo.isEmpty()) {
    floodScore = 60;
    riskFactors.add("Flood zone information pending verification");
} else {
    for (FloodZoneInfoResponse flood : floodZoneInfo) {
        if ("HIGH".equalsIgnoreCase(flood.getFloodRiskLevel())) {
            riskScore += 30;
            floodScore -= 40;
            riskFactors.add("Property is located in a high flood risk area");
        } else if ("MEDIUM".equalsIgnoreCase(flood.getFloodRiskLevel())) {
            riskScore += 15;
            floodScore -= 20;
            riskFactors.add("Property is located in a moderate flood risk area");
        }
        if (Boolean.TRUE.equals(flood.getFloodInsuranceRequired())) {
            riskScore += 10;
            floodScore -= 10;
            riskFactors.add("Flood insurance is required");
        }
    }
}
// ... ownership, tax, environmental and permit signals follow the same
// additive pattern before scores are clamped to [0, 100] and an overall
// HIGH / MEDIUM / LOW band + recommendation string is derived (Section 5.4).

```

10. Appendix A — Complete REST API Reference

Consolidated from every @RestController in the backend module. Base URL in local development: <http://localhost:8080>.

Endpoint	Method	Description
/auth/register	POST	Register a new user
/auth/login	POST	Authenticate and receive a JWT
/api/users	GET	List users
/api/users/count	GET	Total user count
/api/properties	GET	List properties
/api/properties	POST	Create a property
/api/properties/{id}	GET	Get property by id
/api/properties/{id}	PUT	Update property
/api/properties/{id}	DELETE	Delete property (Administrator)
/api/properties/search	POST	Search properties
/api/properties/validate-address	POST	Validate an address
/api/properties/due-diligence	POST	Trigger due-diligence pull
/api/properties/{id}/risk-assessment	GET	Get risk assessment
/api/properties/{id}/comparables	GET	Get comparable properties
/api/properties/{id}/valuation	GET	Get valuation history
/api/property-history/{propertyId}	GET	Get property history
/api/ownership/property/{id}	GET	Get ownership records
/api/property-tax/property/{id}	GET	Get tax history
/api/zoning/property/{id}	GET	Get zoning info
/api/flood-zones/property/{id}	GET	Get flood zone info
/api/permits/property/{id}	GET	Get building permits
/api/environmental/{id}	GET	Get environmental records
/api/reports/{propertyId}	GET	Generate consolidated report
/api/notifications	GET	List notifications
/api/notifications/{id}/read	PUT	Mark notification read
/api/notifications/read-all	PUT	Mark all notifications read
/api/notifications/{id}	DELETE	Delete a notification

Endpoint	Method	Description
/api/notifications/read	DELETE	Clear read notifications
/api/audit	POST	Record an audit log entry
/api/audit	GET	List all audit logs
/api/audit/user/{username}	GET	Audit logs by user
/api/audit/module/{module}	GET	Audit logs by module
/api/audit/action/{action}	GET	Audit logs by action
/api/audit/api	GET	API request/response logs
/api/audit/date-range	GET	Audit logs by date range
/api/audit/report	GET	Aggregated audit report
/api/purchases/property/{id}	POST	Initiate purchase (Buyer)
/api/purchases/my	GET	Buyer's purchases
/api/purchases/property/{id}/status	GET	Purchase status
/api/purchases/legal/pending	GET	Pending legal reviews
/api/purchases/legal/history	GET	Legal review history
/api/purchases/{id}/legal/approve	PUT	Legal approve
/api/purchases/{id}/legal/reject	PUT	Legal reject
/api/purchases/financial/pending	GET	Pending financial reviews
/api/purchases/financial/history	GET	Financial review history
/api/purchases/{id}/financial/approve	PUT	Financial approve
/api/purchases/{id}/financial/reject	PUT	Financial reject
/api/purchases/completed/customers	GET	Completed-purchase customers
/api/dashboard	GET	Role-aware dashboard summary
/v3/api-docs, /swagger-ui/**	GET	OpenAPI / Swagger documentation

11. Conclusion

The delivered Real Estate Due Diligence Agent codebase implements the great majority of the original brief: a ten-module Spring Boot backend on Java 21 with JWT and Google OAuth2 authentication, a five-role RBAC model, a sixteen-table PostgreSQL schema, retry-hardened external-integration client interfaces, a transparent weighted risk-scoring engine, side-by-side property comparison, and a two-stage (legal + financial) purchase-approval workflow that was not explicitly called out in the original brief but is fully built and functioning. The React 19 / Vite / Tailwind frontend covers all five roles with dedicated dashboards and screens, and produces exportable PDF/Excel reports client-side.

Three gaps are worth carrying into the next phase of work, all documented precisely above rather than glossed over: (1) the external data-source clients currently return validated stub data behind real integration contracts — connecting them to genuine Land Registry, Tax Authority, FEMA-style flood and permit/environmental APIs is the main remaining integration task; (2) server-side report rendering (JasperReports/iText) and Redis caching are declared as dependencies but not yet wired into working code, with the frontend covering PDF/Excel export in the meantime; and (3) containerisation, CI/CD and cloud deployment artifacts (Docker, GitHub Actions, Nginx) do not yet exist in the repository, though the application's externalised configuration (datasource URL, JWT secret, OAuth2 credentials, frontend URL) means the path to adding them is straightforward and is outlined in Section 8.3.

Overall, the platform's core value proposition — pulling scattered public-record data into one risk-scored, auditable, role-gated due-diligence workspace — is implemented and demonstrable end-to-end today, from address entry through to a downloadable report and a fully-tracked purchase approval.